

Demand Paging

Ivy Peng

Associate Professor

EECS, KTH



Outline

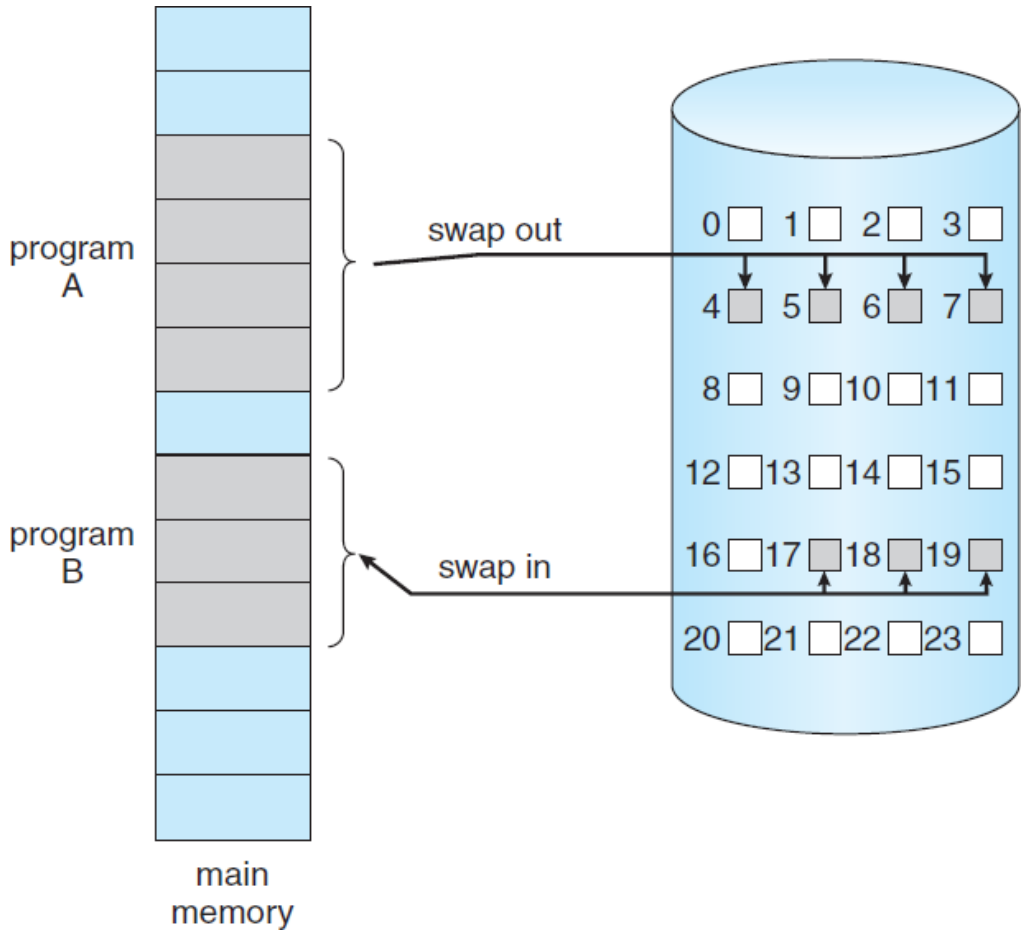
- **Demand Paging**
- Page Replacement / Eviction Policies
  - FIFO
  - Optimal
  - Counting
  - LRU
  - LRU-Approximation
- Thrashing

By the End of This Lecture

- Q1 What happens on a page fault, step by step?
- Q2 Compare page-replacement algorithms?
- Q3 What is thrashing, and how does the working-set model prevent it?

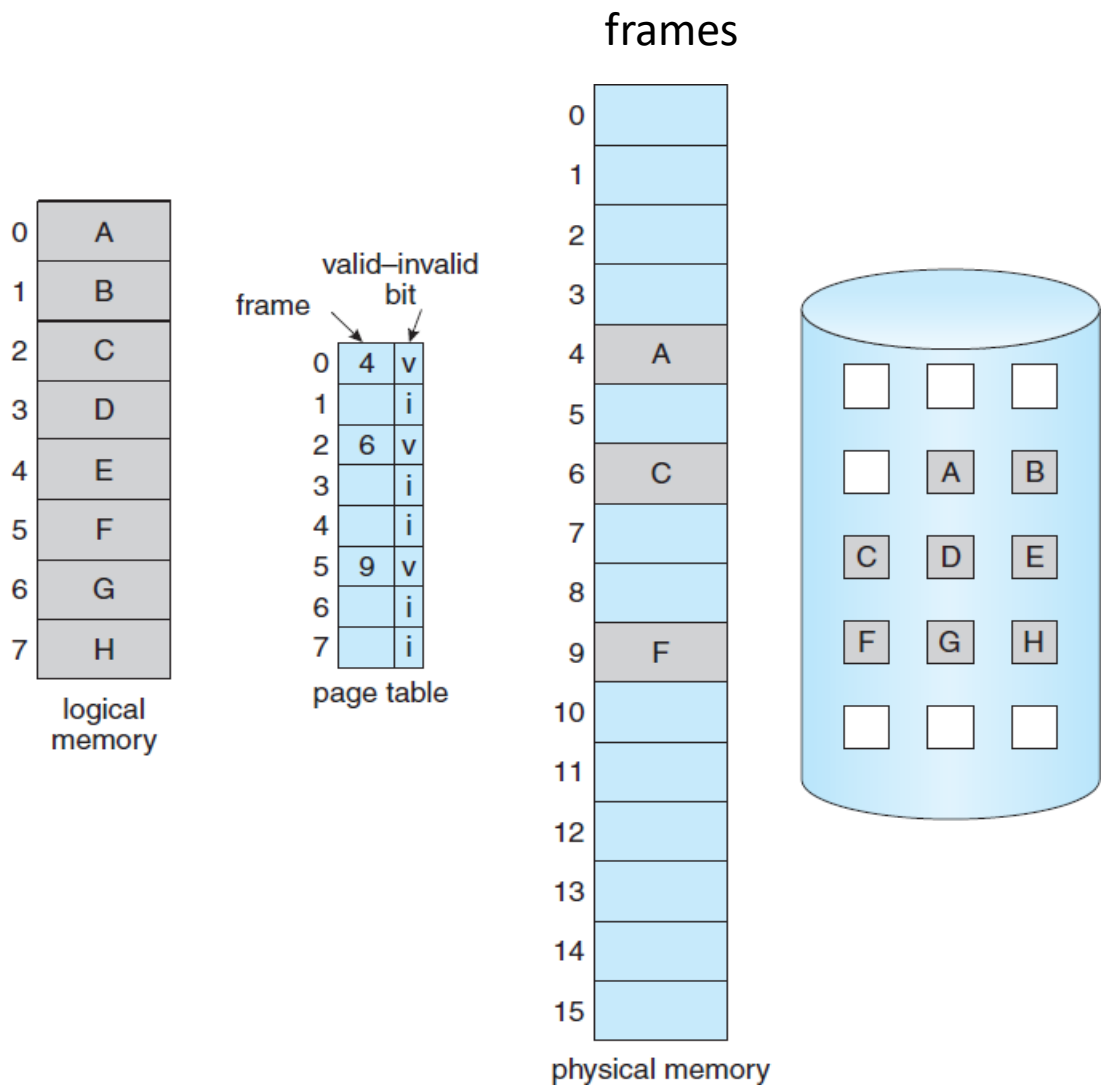
Demand Paging

- Only bring a page into memory when it is needed – on-demand



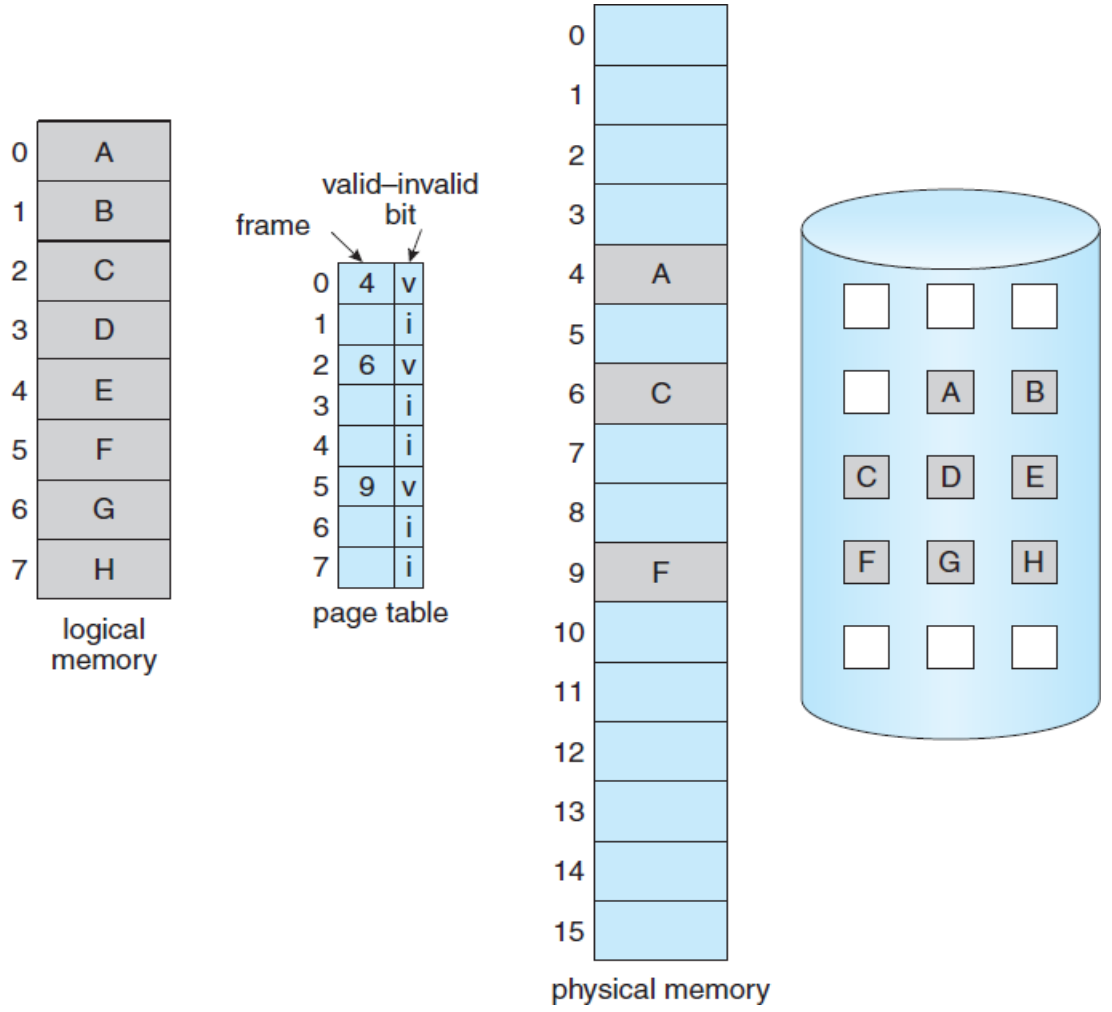
Basic Concepts

- Keep track of which pages are in use, and only bring those pages in memory
- ***Valid-invalid bit***: distinguish between the pages in memory and on disk.
- v: memory resident
- i: not in memory



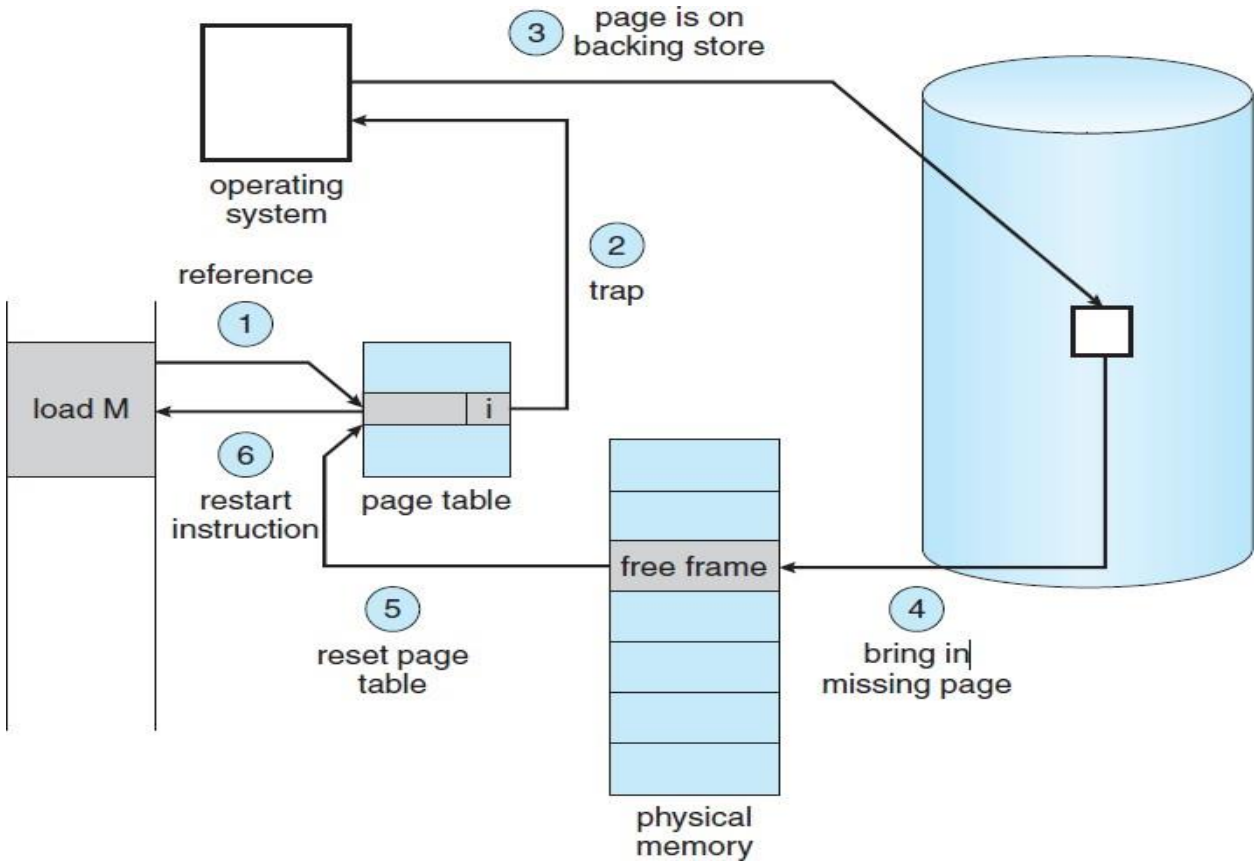
Page Fault

- Access to a page marked **invalid** causes a page fault.
- A page fault causes a trap to the OS - **Page Fault Handling**
  - Check access right to the page
  - Bring the desired page from disk into memory



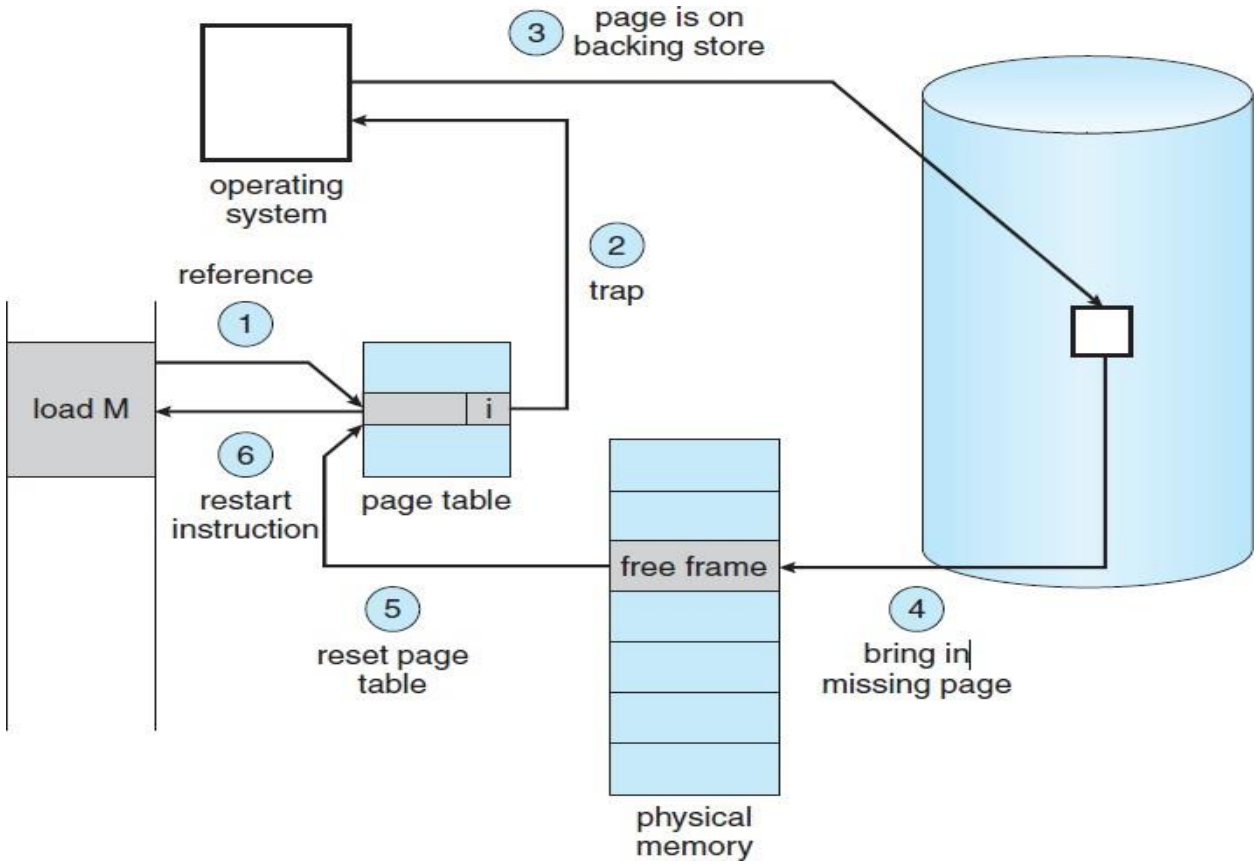
Page Fault Handling (1/3)

- First, check the process's page table to determine whether the reference was a **valid** or an **invalid** memory access.
  - If the reference was invalid, we terminate the process.



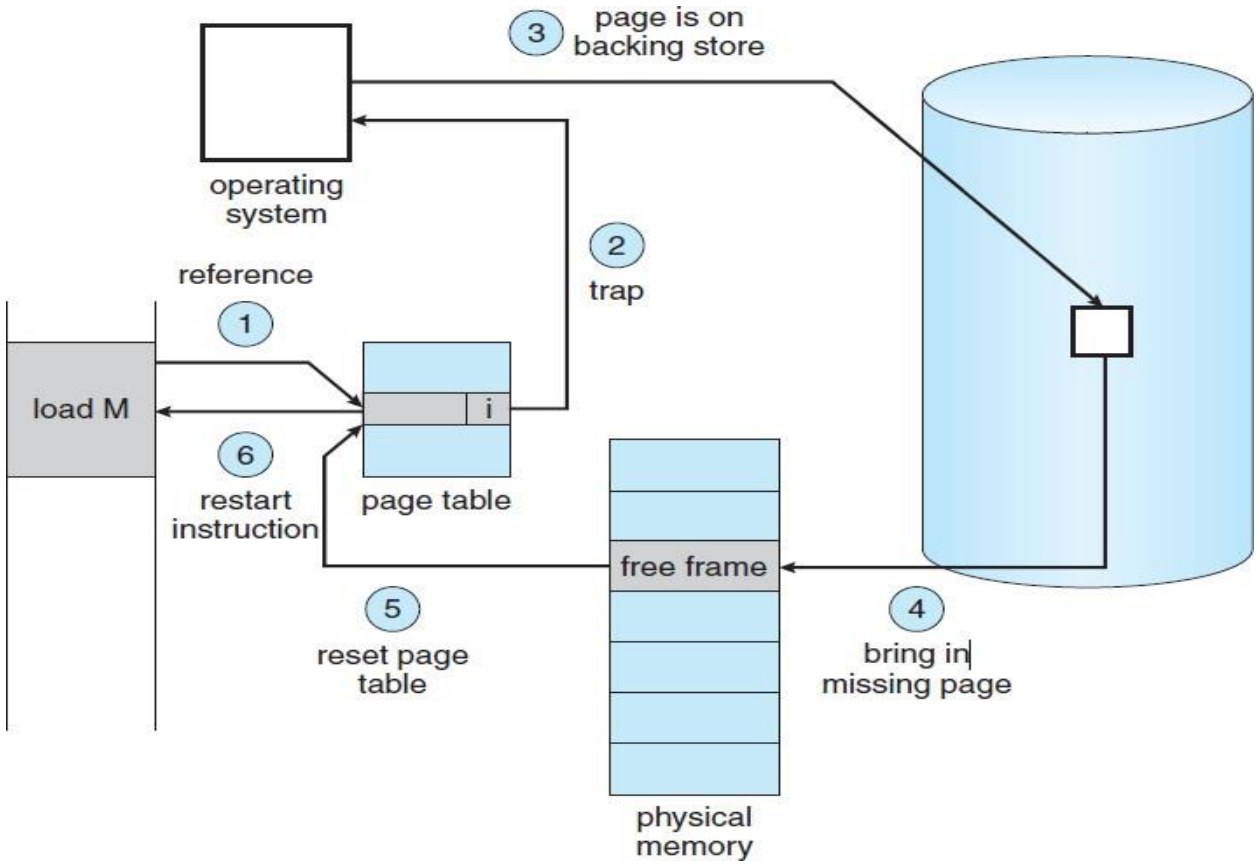
Page Fault Handling (2/3)

- If the reference is **invalid**, need to **page in:**
  - find a free frame in physical memory
  - schedule a disk operation to read the desired page from storage into the newly allocated frame



Page Fault Handling (3/3)

- When the disk operation finishes, modify the page table to indicate that the page is in memory.
- Restart the instruction that was interrupted by the trap – e.g, ‘replay the load M’



Page Faults are Expensive

- Let p be the probability of a page fault ($0 \leq p \leq 1$).
- Assume a typical memory access time is 100 nanoseconds
- Page fault time is much higher, e.g., 8 milliseconds
  - Page fault interrupt
  - Service the page fault: mapping, read the page into memory
  - Restart the process
- We can model **Effective Access Time** = $(1 - p) \times T_{mem_access} + p \times T_{page_fault}$, p is the rate of page faults
  - *What is effective access time if p = 0.1% ?*
  - *What is effective access time if p = 0.001% ?*

The Cost of a Page Fault

- A resident access takes ~100 ns; servicing a page fault (a disk read) takes ~8 ms: about 80,000× slower.
- Effective Access Time (EAT) = $(1 - p) \times T_{mem} + p \times T_{fault}$, p = fault rate

Fault rate p	Effective Access Time	Slowdown
0	100 ns	1×
0.1%	0.999·100 + 0.001·8,000,000 = 8099.9 ns	≈ 81×
< 1 fault per 800K accesses	need EAT ≤ 110 ns	**for ≤10% slowdown**

- One page fault per thousand accesses makes the program ~80× slower.
- To keep the slowdown under 10%, faults must be rarer than about one in 800,000.

Measure Page Faults

- Option 1: Perf in Linux
  - Demo

```
ip2@ip2-QEMU-Virtual-Machine:~$ perf stat  -e major-faults,minor-faults hostname
ip2-QEMU-Virtual-Machine

 Performance counter stats for 'hostname':

                 0      major-faults
                44      minor-faults

       0,001124482 seconds time elapsed

       0,000000000 seconds user
       0,001093000 seconds sys
```

- Option 2: use ‘time’ utility tool

```
ip2@ip2-QEMU-Virtual-Machine:~$ /usr/bin/time sleep 3
0.00user 0.00system 0:03.00elapsed 0%CPU (0avgtext+0avgdata 1792maxresident)k
64inputs+0outputs (1major+82minor)pagefaults 0swaps
```

Exposing Page Faults

- Implement your own page fault handler by specifying the function to run on SIGSEGV

- **#include <signal.h>**
- **int sigaction(int *signum*, const struct sigaction *_Nullable restrict *act*,**
  struct sigaction *_Nullable restrict *oldact*);

```
struct sigaction {
    void     (*sa_handler)(int);
    void     (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;
    int        sa_flags;
    void     (*sa_restorer)(void);
};
```

https://man7.org/linux/man-pages/man2/sigaction.2.html

Alternative Usage of fault handler

- **mprotect + sigaction**: using mprotect to make a region inaccessible, trigger a fault, calling a user-specified signal handler

- Many interesting use cases:
  - Object-oriented databases
    - Bring in objects on demand
    - Keep track of which objects may be dirty
    - Manage memory as a cache for much larger object DB
  - Snapshot processes (copy on write)
  - Trace page usage
  - Emulate new instructions

```
#include <signal.h>
#include <sys/mman.h>
#include <unistd.h>

static char *page;                  // the protected page
static long  pagesize;

static void handler(int sig, siginfo_t *si, void *ucontext) {
   (void)sig; (void)ucontext;
   printf(" [handler] fault at address %p\n", si->si_addr);

   //make the page writable
   mprotect(page, pagesize, PROT_READ | PROT_WRITE);
}

int main(void) {
   pagesize = sysconf(_SC_PAGESIZE);

   // 1. Register the handler for SIGSEGV
   struct sigaction sa;
   sa.sa_flags       = SA_SIGINFO;
   sa.sa_sigaction = handler;
   sigemptyset(&sa.sa_mask);
   if (sigaction(SIGSEGV, &sa, NULL) == -1) { perror("sigaction"); exit(1); }

   page = mmap(NULL, pagesize, PROT_READ | PROT_WRITE,
            MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
   if (page == MAP_FAILED) { perror("mmap"); exit(1); }

   if (mprotect(page, pagesize, PROT_READ) == -1) { perror("mprotect"); exit(1);
}

   page[0] = 42;                    // trigger SIGSEGV

   munmap(page, pagesize);
   return 0;
}
```



Outline

- Demand Paging
- **Page Replacement / Eviction Policies**
  - FIFO
  - Optimal
  - Counting
  - LRU
  - LRU-Approximation
- Thrashing

The Free-Frame List

- The OS maintains a list of free frames in physical memory for the system
  - Use a linked list to maintain a pool of free frames for allocation
  - Zero-fill-on-demand technique zero-out a frame's content before being allocated
- When free frames run low, **memory reclamation** may be needed



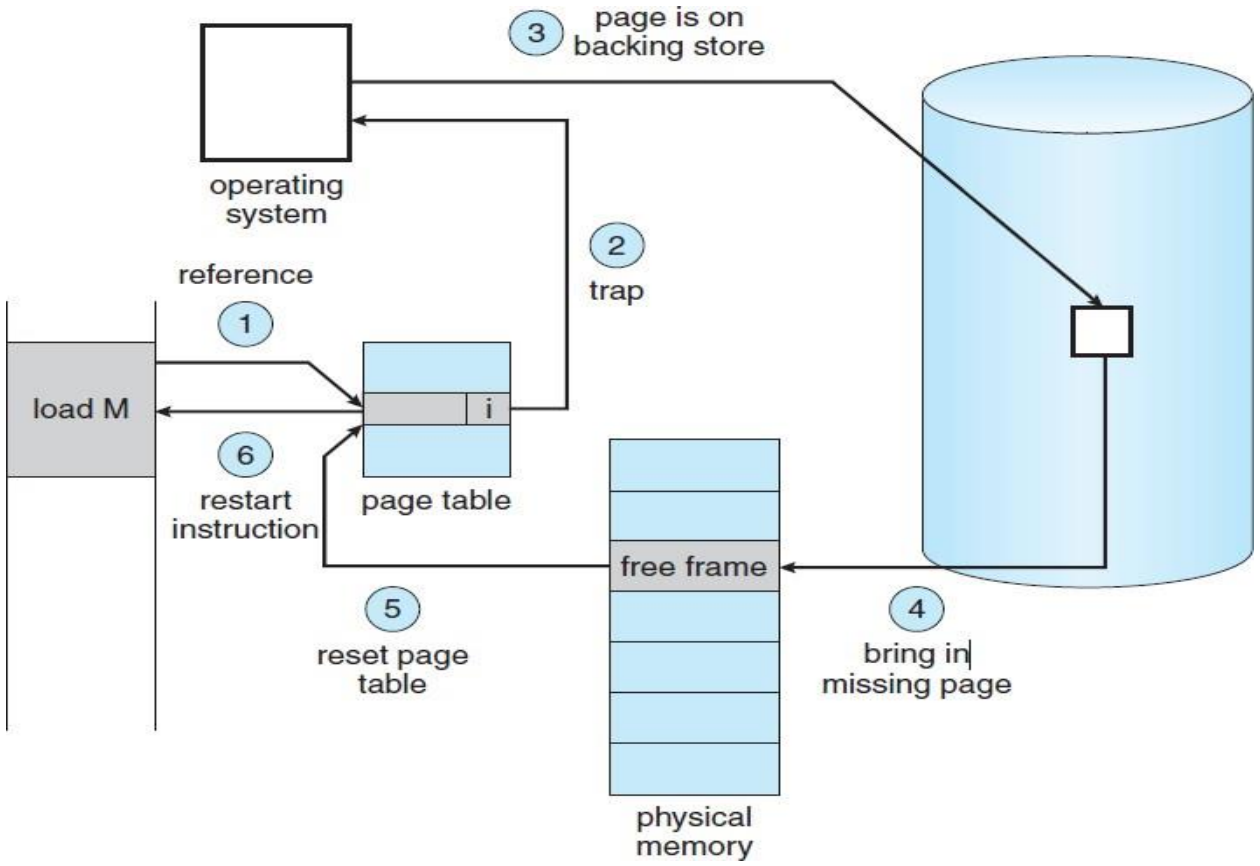
A List of free frames

Modern Memory Reclamation: kswapd and Watermarks

- When frames run low, the OS reclaims pages via the active/inactive LRU lists.
- Linux uses three **watermarks**: min, low, high, as threshold levels of free memory
  - Reclamation stops if Free Memory is high
  - Free Memory drops below low → wake `**kswapd**` (background reclaim) to refill up to high.
  - Free Memory hits min → direct reclaim: the allocating thread reclaims synchronously.
- Reclaim writes dirty pages back and drops clean ones, refilling the free-frame list.

When no free frame is left

- If the reference is valid, need to **page in** – find a free frame in physical memory
- Schedule a disk operation to read the desired page from storage into the newly allocated frame



Memory Over-allocation

- Assume, we had 40 frames in physical memory.
- And, we run 6 processes
  - each is 10 pages in size, but only actually uses 5 pages.
- By Demand Paging, how many frames are free?
- Without demand paging, we won't be able to support 6 processes on the system

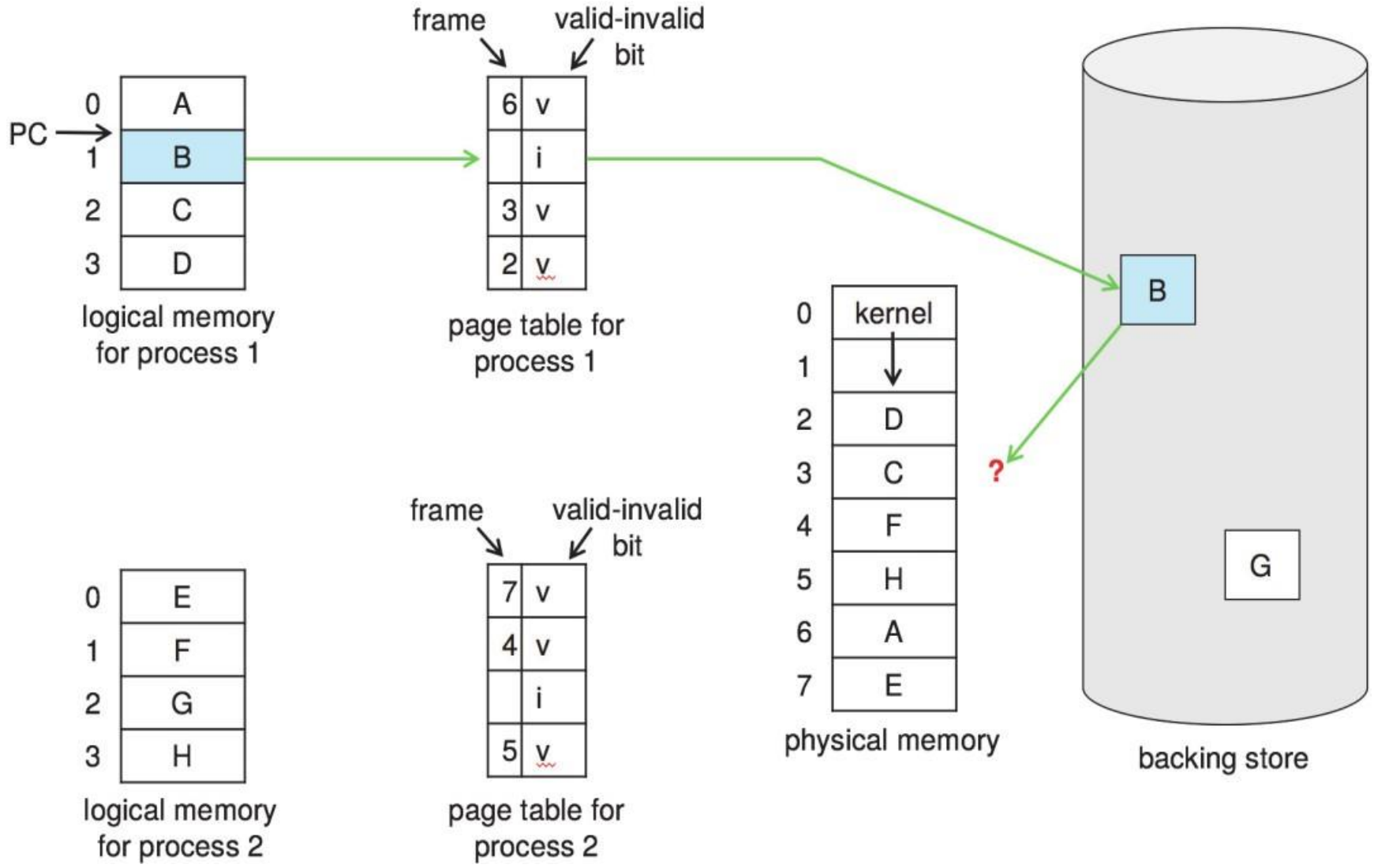


- At one point, it is possible that all these 6 processes try to use all 10 pages – need 60 frames in physical memory

Memory Over-allocation

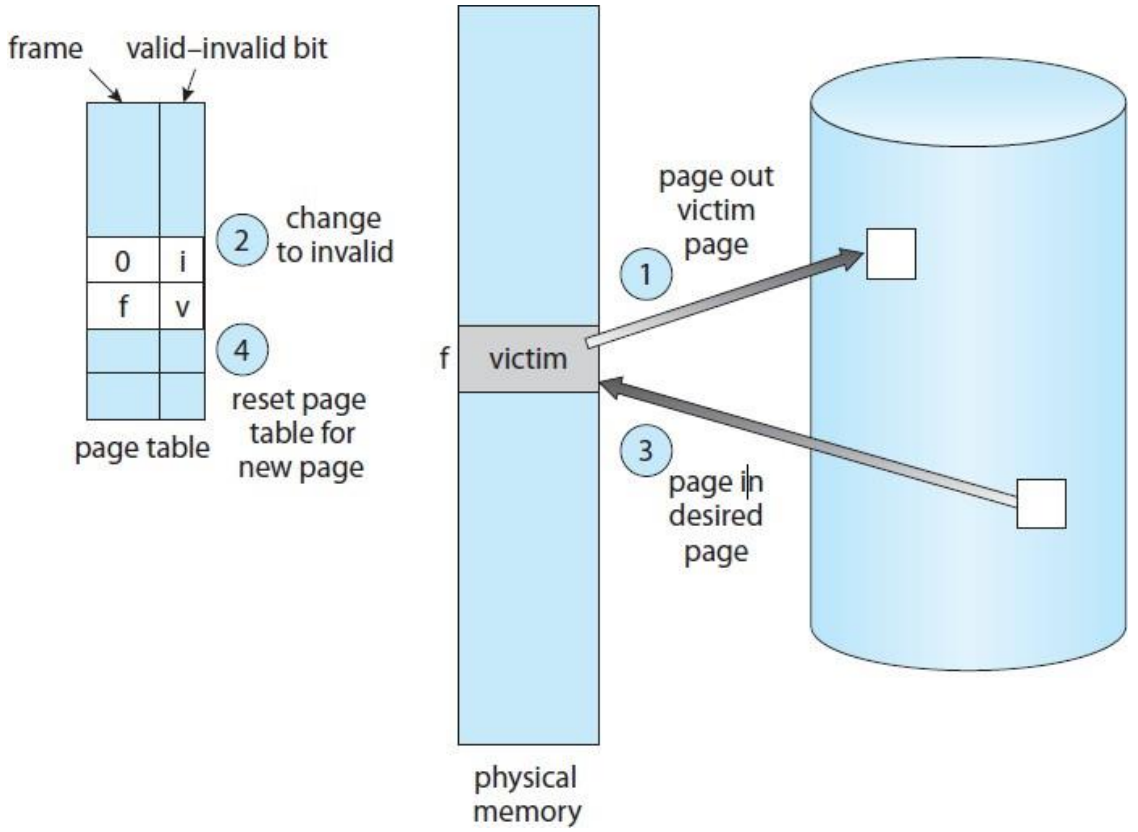
- While a user process is executing, a page fault occurs.
- The OS determines where the desired page is residing on the disk.
- But, it finds that there are no free frames on the free-frame list.
- Need for page replacement

Need for Page Replacement



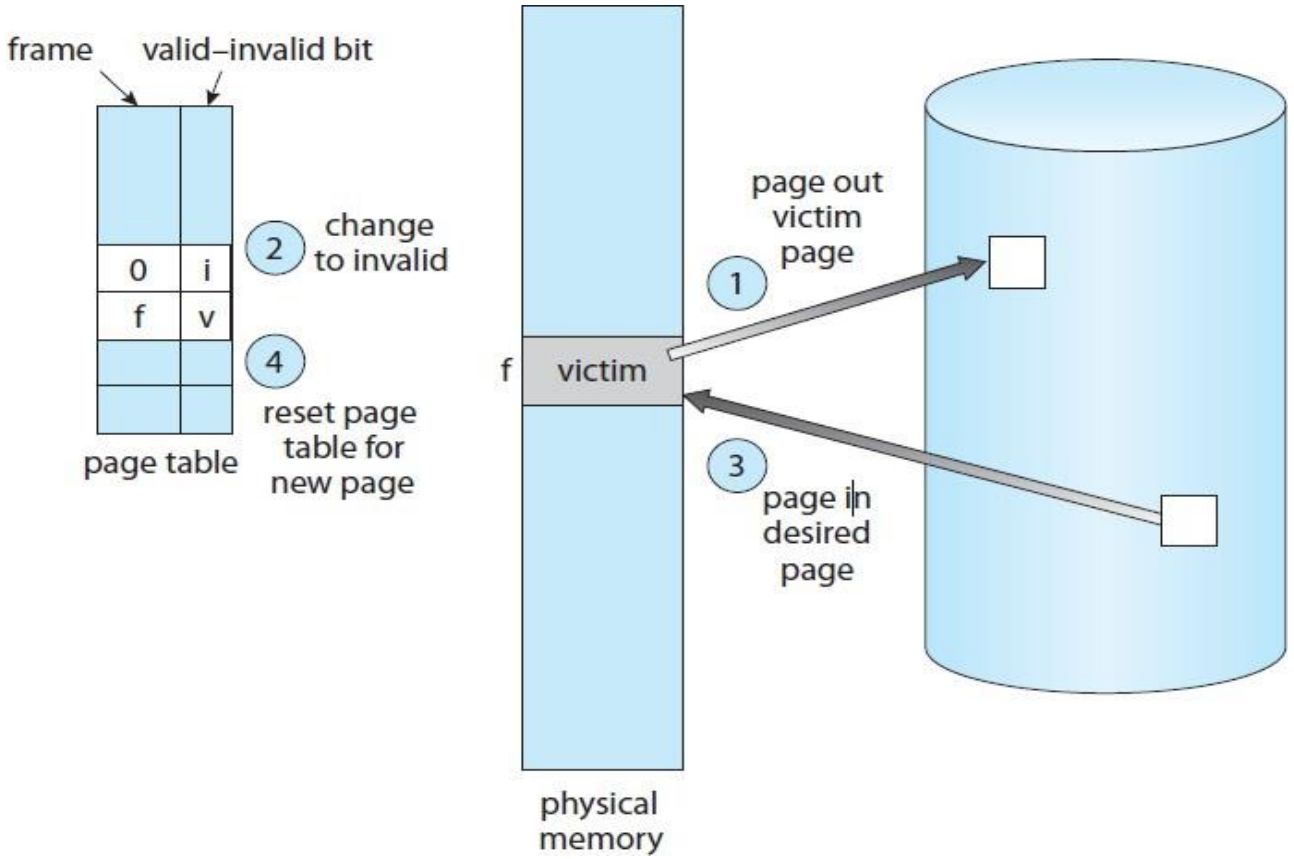
Page Replacement

- If there is no free frame, use a page replacement algorithm to select a **victim frame**
  - If the victim page is dirty, write victim frame to disk.
- Bring the desired page from storage into the (newly) free frame in physical memory
- Update the page and frame tables



Dirty Bit

- Use modify (dirty) bit to reduce overhead of page transfers - only dirty pages are written to disk.



The Free-Frame List for N Processes

- The OS maintains a list of free frames in physical memory for the system - Global
- Multiprogramming means multiple process are running on a system and they compete for free frames

- How to allocate N free frames among M processes, each requesting different number of frames?
  - Equal allocation: N/M to each process
  - Proportional allocation to each process's virtual memory size: N * S_i/S_{total} for process I
  - Problems?
    - Actual usage, priority

- For page replacement, algorithms can be either Global replacement or Local replacement

Global vs. Local Allocation

- Global replacement: a process can select a replacement frame from all frames
  - Each frames that are assigned to other processes
  - Performance depends on other processes at runtime
- Local replacement: a process can ONLY select from its own set of allocated frames

- Global replacement performs better, is more common



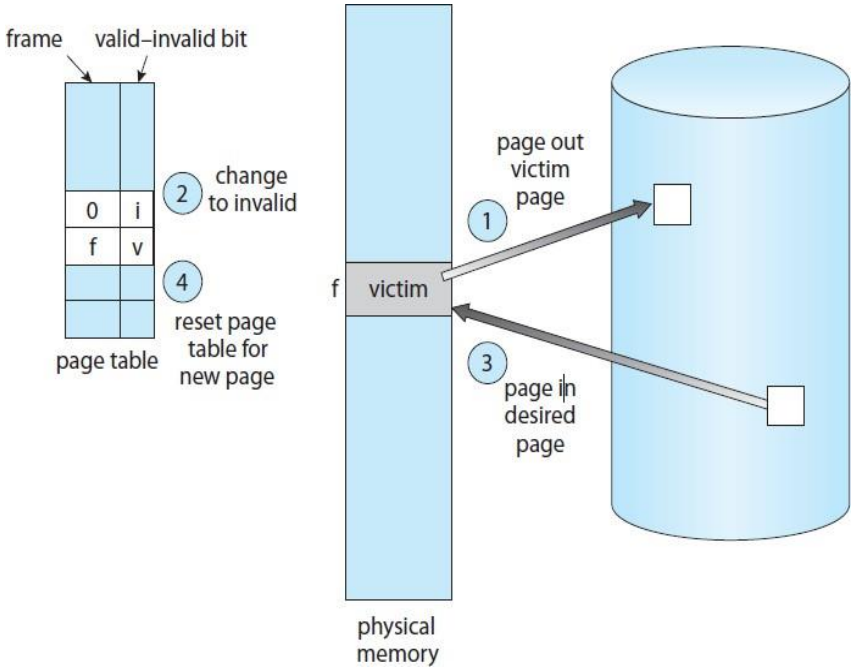
Outline

- Demand Paging
- Page Replacement / Eviction Policies
  - **FIFO**
  - **Optimal**
  - **Counting**
  - **LRU**
  - **LRU-Approximation**
- Thrashing

Page Replacement Algorithms

- Find **Victim Pages** using different algorithms:
  - First-In-First-Out (FIFO)
  - Optimal
  - Least Recently Used (LRU)
  - LRU-Approximation
  - Counting-Based

- The **number of page faults** is a good metric for evaluating these algorithms
  - Given a reference string as a sequence of page numbers
    - {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1}
  - Evaluate each algorithm by running it on the reference string
  - Compare the number of page faults generated on the string.

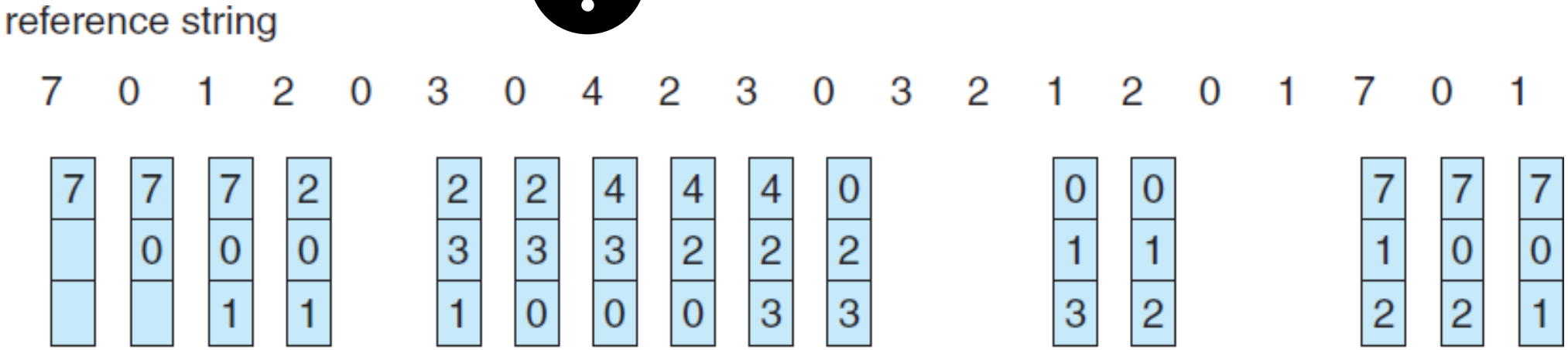


FIFO Page Replacement

- Reference string: {7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1}
- 3 frames (3 pages can be in memory at a time per process)

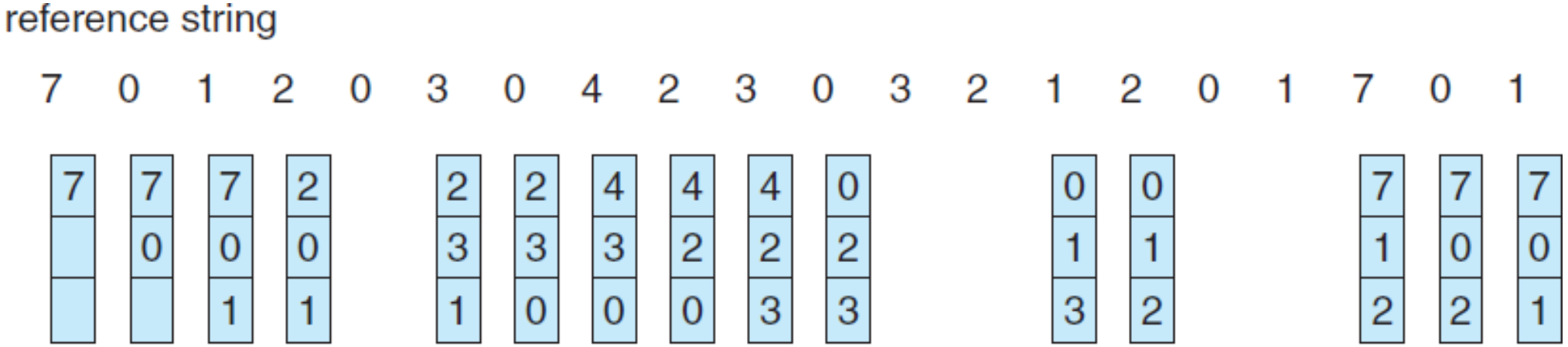


Which references will cause page faults? How many faults?



FIFO Page Replacement

- Reference string: {7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1}
- 3 frames (3 pages can be in memory at a time per process)



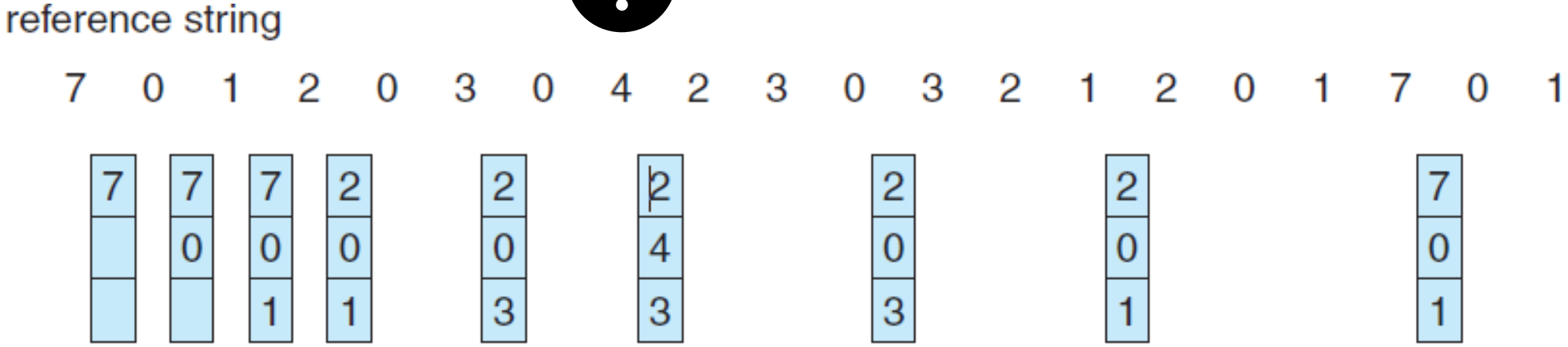
- 15 page faults

Optimal Page Replacement

- Replace page that **will not be used for longest period of time**.
- How do you know this? Can't read the future! Instead, use it as an ideal case for measuring how well your algorithm performs.

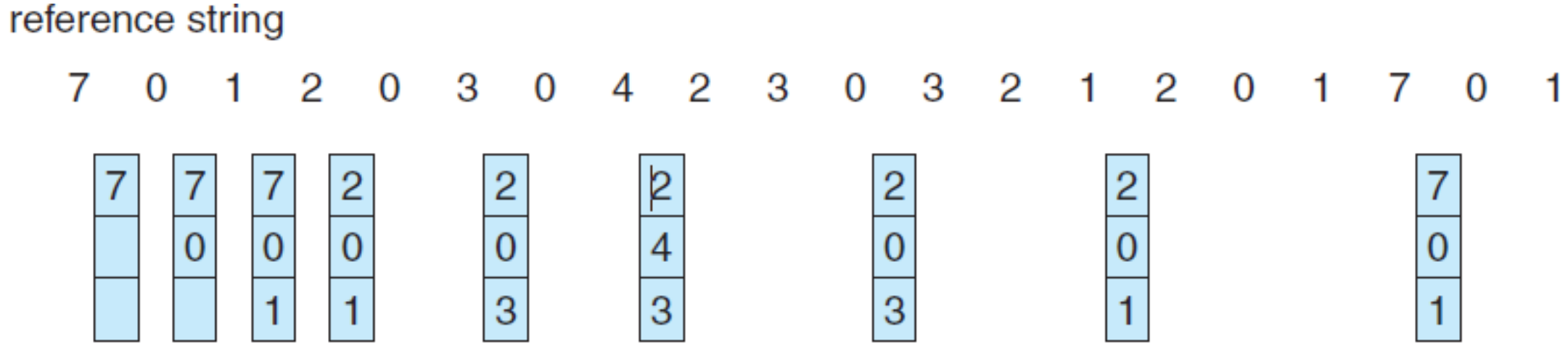


Which references will cause page faults? How many faults?



Optimal Page Replacement

- Replace page that will **not be used for longest period of time**.
- How do you know this? Can't read the future! Instead, use it as an ideal case for measuring how well your algorithm performs.



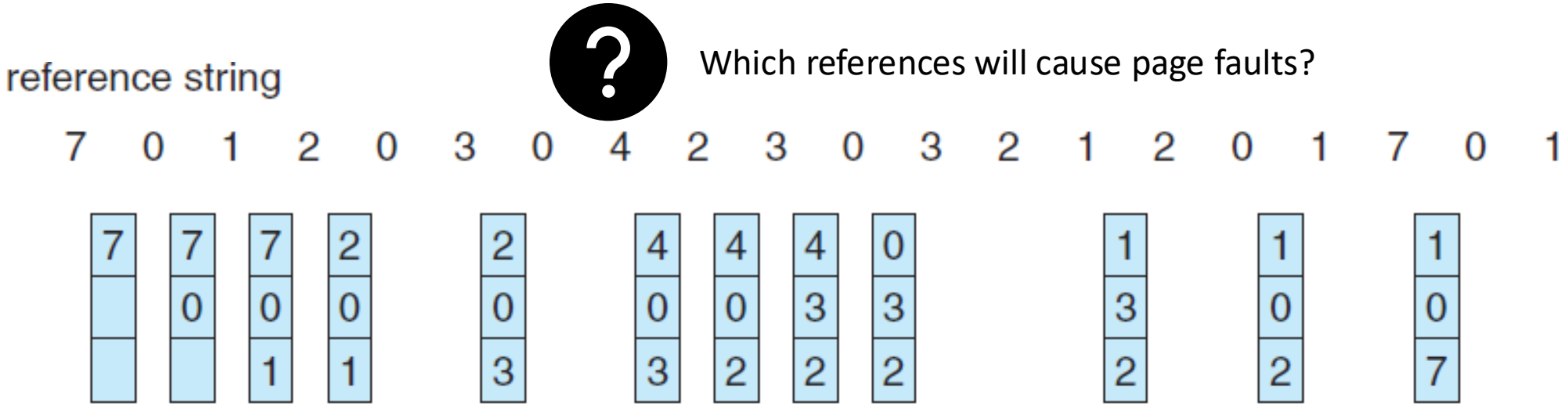
- 9 faults

Counting Page Replacement

- Keep a counter of the number of references that have been made to each page.
- Least Frequently Used (LFU) algorithm: replaces page with smallest count.
- Most Frequently Used (MFU) algorithm: based on the argument that the page with the smallest count was probably just brought in and has yet to be used.
- What are potential limitations?

Least-Recently Used (LRU)

- Use past knowledge rather than the future.
- Replace page that has not been used in the most amount of time.
- Generally good and widely used



LRU Implementation (1/2)

- Option 1: Counter implementation

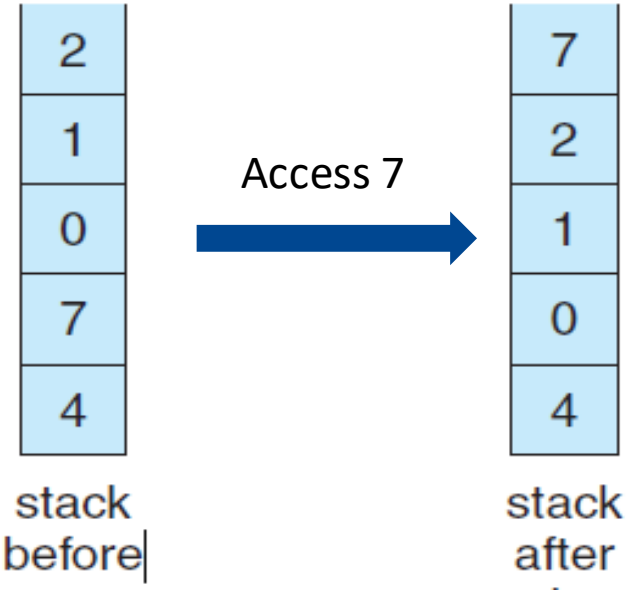
- Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter.

- When a page needs to be changed, look at the counters to find smallest value.

- Time Complexity: search through the whole table.

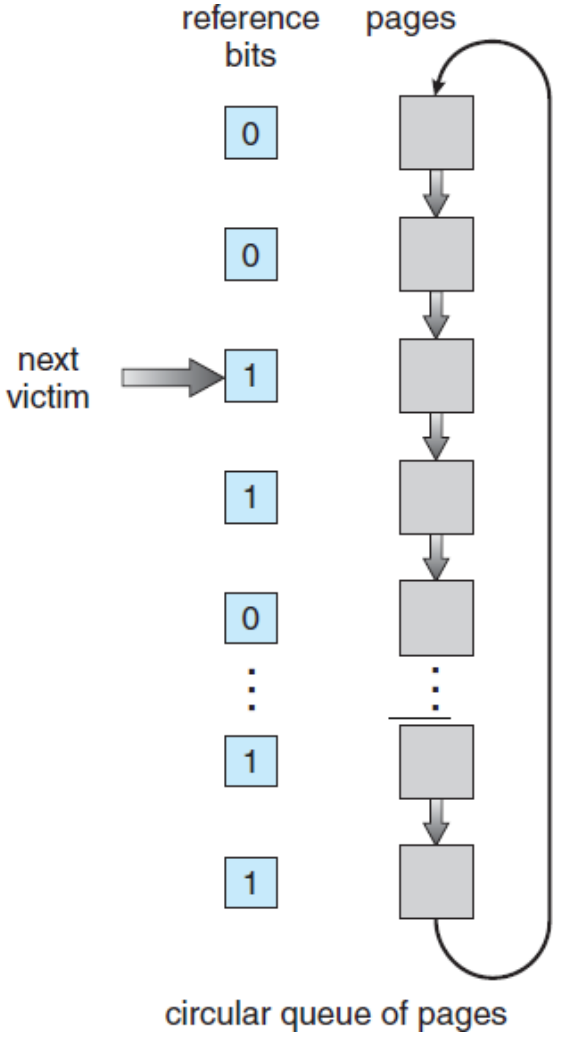
LRU Implementation (2/2)

- Option 2: Stack implementation
  - Use of a stack to record most recent page references.
- Keep a stack of page numbers.
- Page referenced: move it to the top
- No search for replacement.



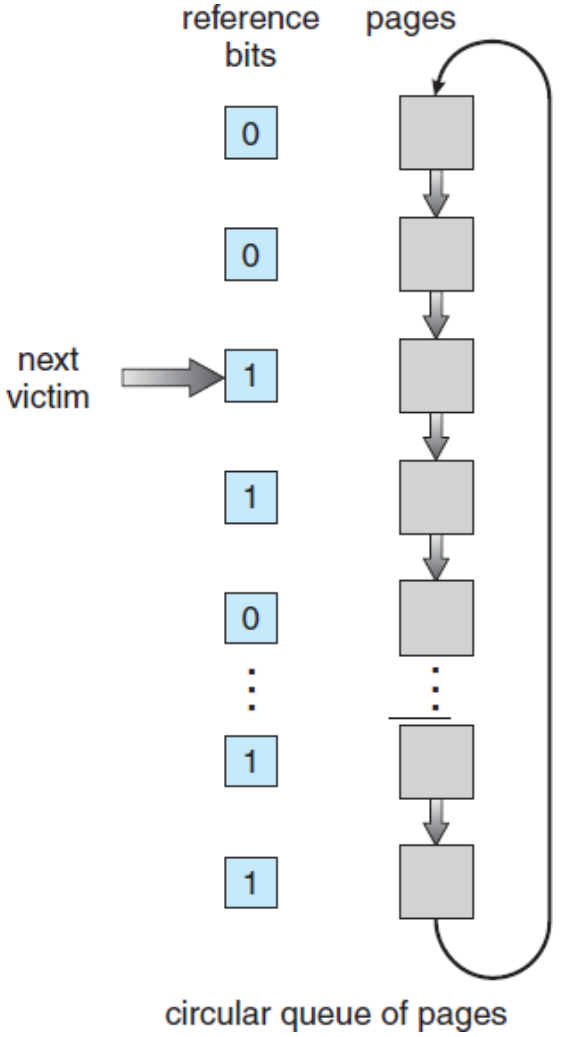
LRU-Approximation

- LRU needs special hardware and still slow
- Improvements: LRU-Approximation
- Each page has a Reference bit
- Second-chance (Clock) algorithm
  - Consider pages in a **round-robin matter** (with circular queue)
  - Keep a pointer to the last considered page
  - See Linux's implementation in slides 44&45



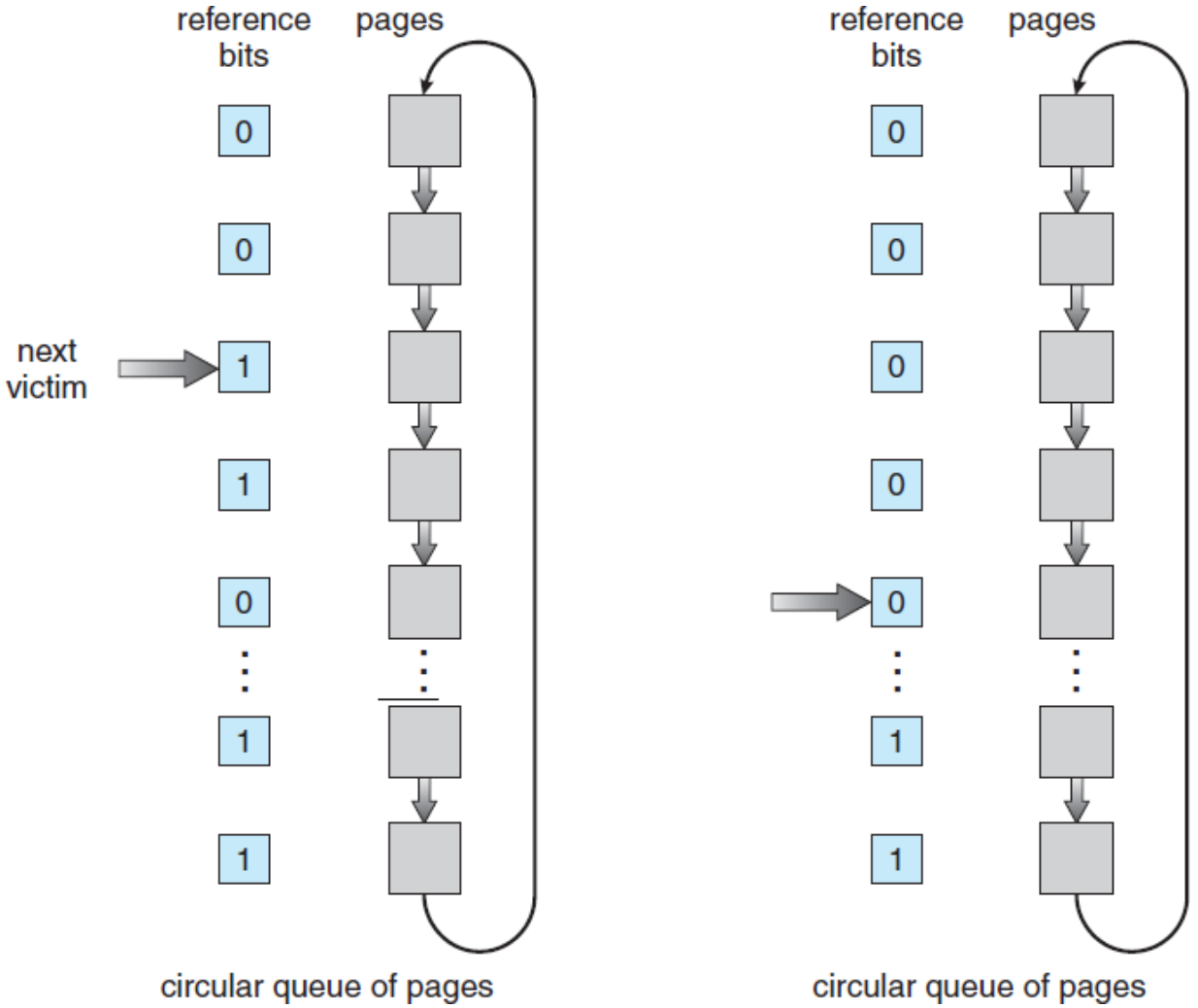
LRU-Approximation

- Each page is associated to a Reference bit, initially = 0
- When page is referenced, set the bit to 1
- During Second-chance (Clock) algorithm, when considering the next page
- If its reference bit = 0, replace it -> victim page
- If its reference bit = 1, reset it to 0 and move on the consideration to the next page – Give it a second chance until the next time it is considered



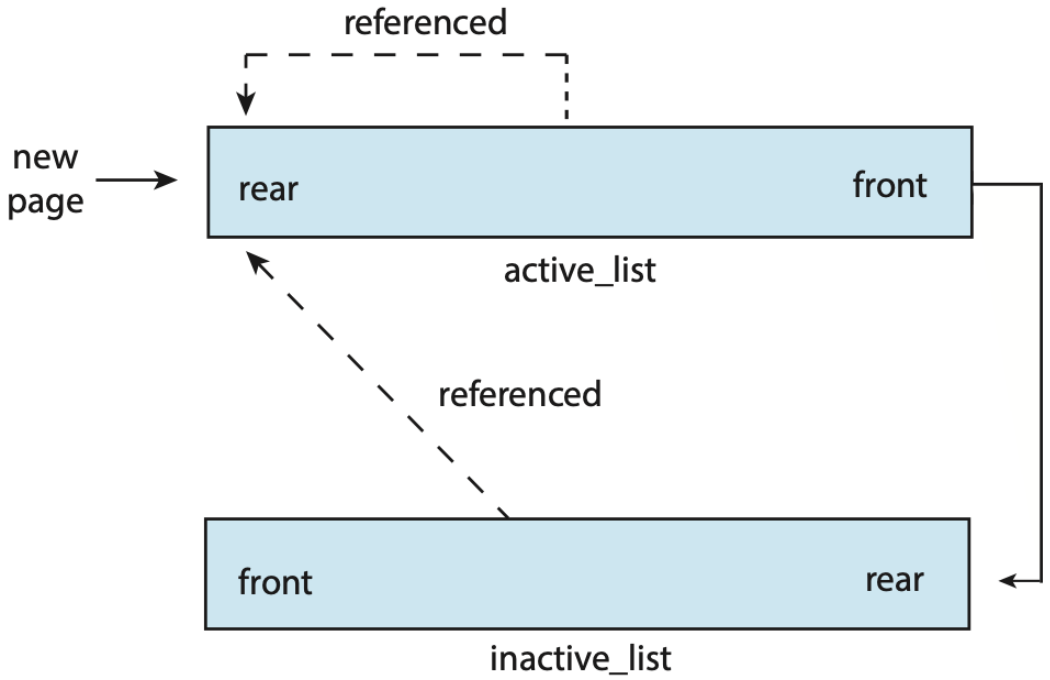
Second-chance/Clock Algorithm

- Generally, it's a FIFO, plus hardware-provided reference bit
- Keep a pointer to the last considered page
- If a page has
  - Reference bit = 0 → replace it
  - Reference bit = 1 then, set reference bit 0, leave page in memory, and increment the pointer to the next page



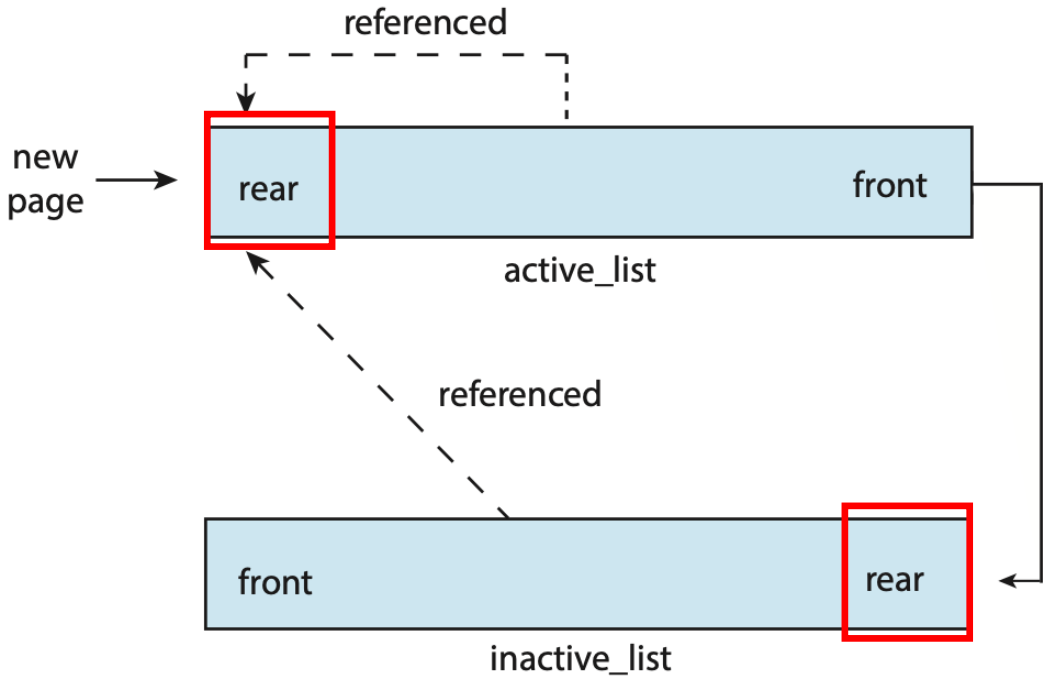
Active and Inactive List (1/2)

- Linux uses a global page-replacement policy like LRU-approximation clock algorithm
- Linux maintains two lists of pages: an **active** list and an **inactive** list
  - Active pages are those in use
  - Inactive pages are those may be reclaimed



Active and Inactive List (2/2)

- When a page is newly created or referenced, its access bit is set to 1, and is moved to the rear end of active list
- Periodically, the access bit for pages is reset
- Over time, pages at the front end of active list (likely have access bit=0) may be moved to the inactive list.



Belady's Anomaly: More Frames, More Faults

- Intuition: more memory means fewer page faults. But not always.
- Reference string 1 2 3 4 1 2 5 1 2 3 4 5 using FIFO:
  - 3 frames → 9 faults.
  - 4 frames → 10 faults.
  - More frames, more faults: **Belady's anomaly**
- Inclusion Property: if the set of pages held in N frames is always a subset of the set held in N+1 frames
  - Such an algorithm is a Stack Algorithm
- LRU and Optimal satisfy the inclusion property:
  - the pages kept with more frames always **include** those kept with fewer.
  - So adding a frame can **never** add a fault.

Belady LA, Nelson RA, Shedler GS. An anomaly in space-time characteristics of certain programs running in a paging machine. Communications of the ACM. 1969 Jun 1;12(6):349-53.

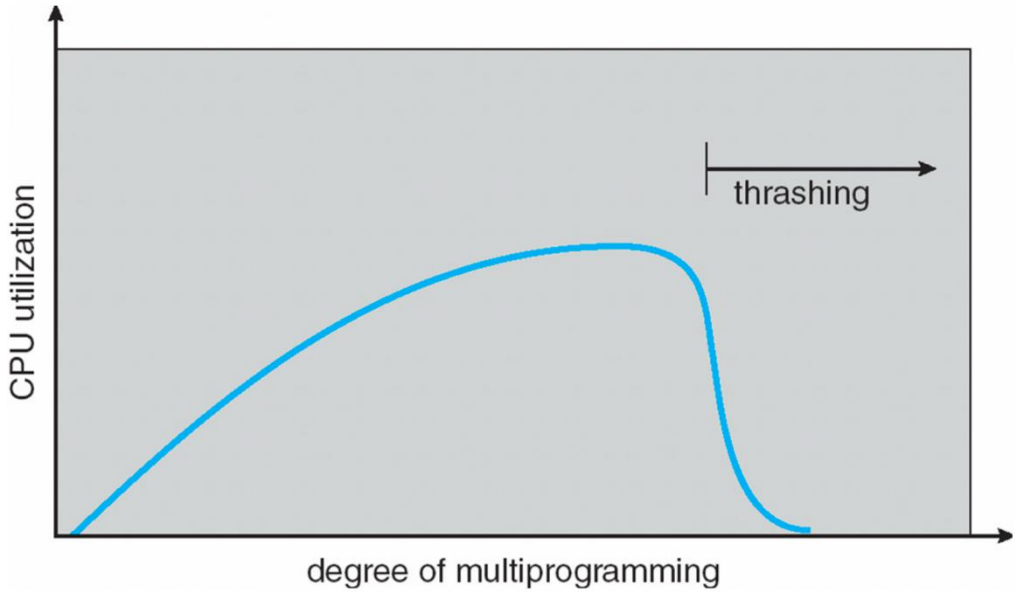


Outline

- Demand Paging
- Page Replacement / Eviction Policies
  - FIFO
  - Optimal
  - Counting
  - LRU
  - LRU-Approximation
- **Thrashing**

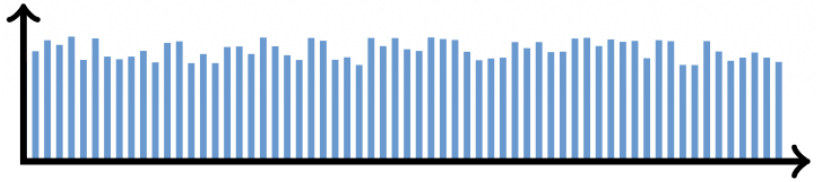
What is Thrashing

- Processes require more memory than the system has
- Whenever a page is brought in, another page, which will be referenced soon, needs to be evicted to make space
- Processes will spend all of their time blocked, waiting for pages to be fetched from disk
- CPU utilization is low but disk utilization is high
- System is not getting much useful work done



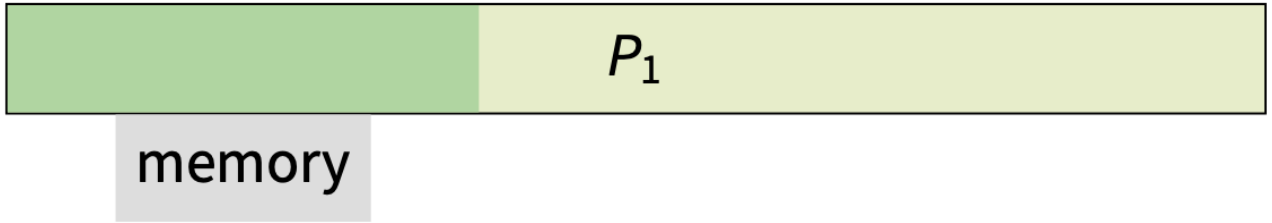
What Causes Thrashing

- Access pattern has no temporal locality (reuse is low)

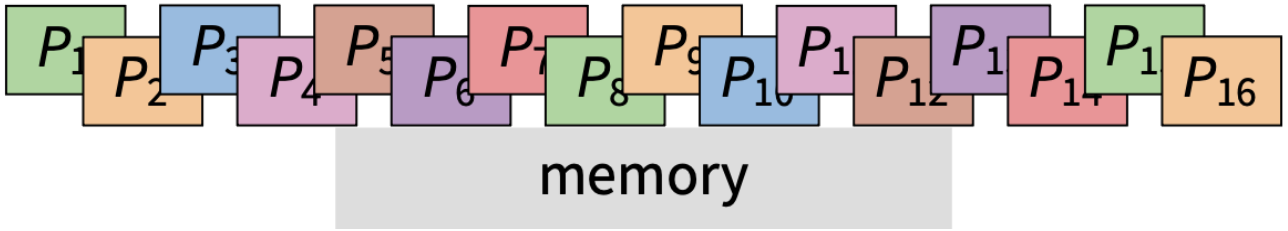


(80/20 rule has broken down)

- Hot memory does not fit in physical memory

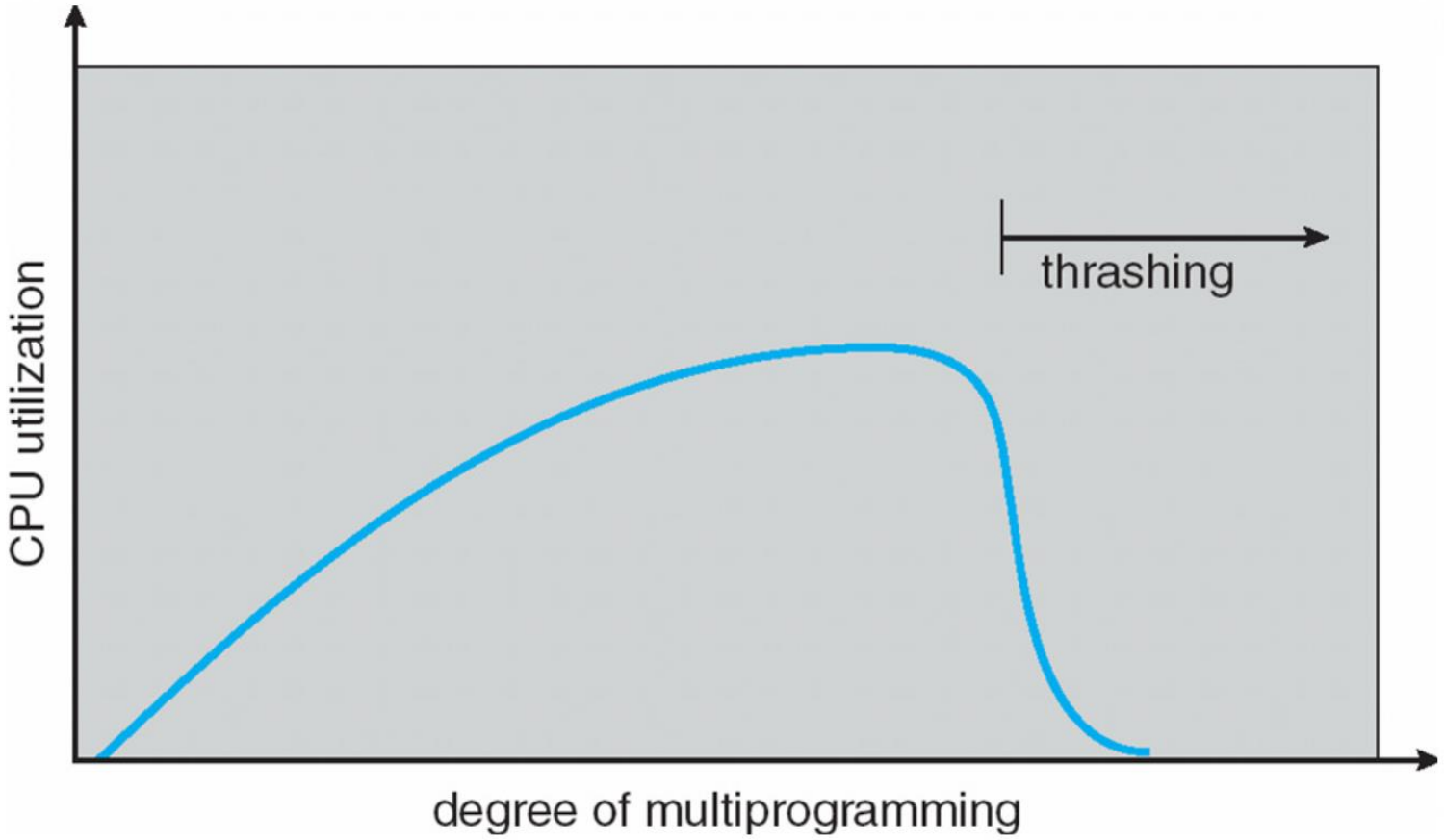


- Too many processes for the system.



Multiprogramming & Thrashing

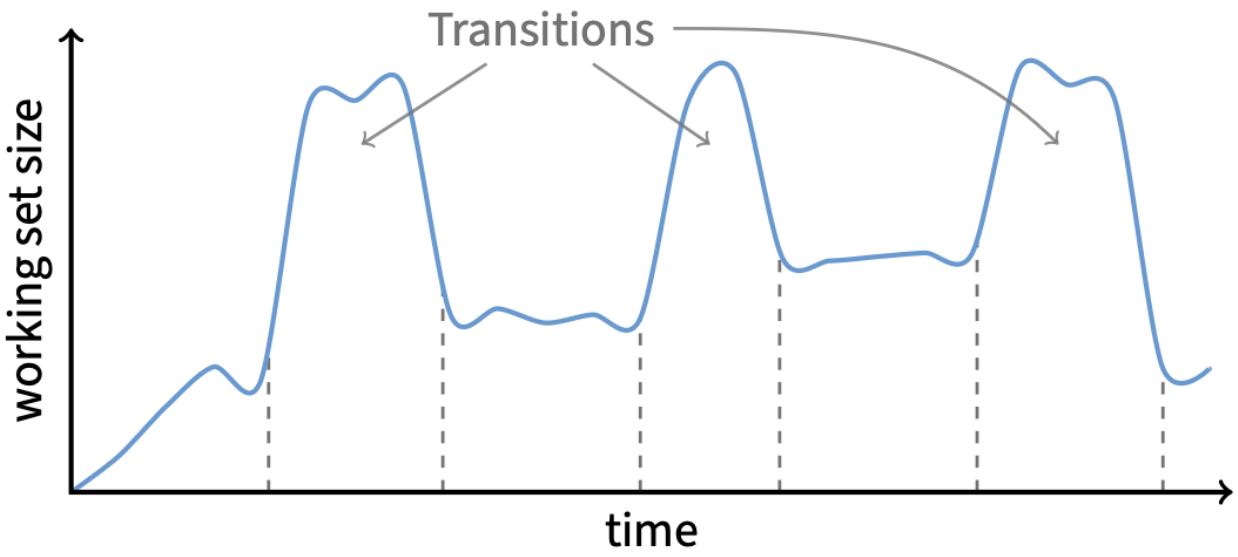
- When too many processes crowd into memory, each one gets too few frames to hold its **working set**.



The Working Set of a Process

- Quantify as the working set W(t, Δ) at time t as the set of **distinct pages referenced** in the last Δ memory references.
- For example, if Δ = 10,000 references and in the last 10,000 references the process touched unique pages {3, 7, 8, 15, 22}, its working-set size = 5.

- It's the minimum memory that a process needs now to run without excessive page faulting/thrashing
- View thrashing from a caching perspective: given locality of reference, how much memory does the process need to make real progress? Only run processes whose memory requirements can be satisfied



Working Set changes during an execution in phases

How to get the Working Set

- Ideally, working set = all pages that process will access **in next T time**. Need prediction.

- Approximate by measuring past. So working set ≈pages accessed in last T time
  - Keep idle time for each page
  - Periodically scan all resident pages in system
    - Access bit set? Clear it and clear the page's idle time
    - Access bit clear? Add CPU consumed since last scan to idle time
    - Working set is pages with idle time < T

Back to Questions

- Q1 What happens on a page fault, step by step?
  - The workflow diagram of page fault handing
- Q2 Compare page-replacement algorithms?
  - LRU, LRU approximate, OPT, FIFO
- Q3 What is thrashing, and how does the working-set model prevent it?
  - a process spends more time paging than computing when its working doesn't fit in its frames.
  - Fix: give each process enough frames for its working set, or reduce the degree of multiprogramming.